

Informe de Testing Aplicado al Modulo Seguridad

Proyecto Piscina API - Aplicaciones Web II

Inicio de la exposicion

Mi modulo es Seguridad. En este modulo trabaje pruebas para guardavidas y accesos. Hice pruebas en tres capas: service, handler y storage. La idea fue comprobar reglas de negocio sin danar la base real del proyecto.

No estoy probando solo getters. Estoy probando reglas reales: que no se cree un acceso invalido, que una ruta protegida responda 401 sin token y que el repositorio con GORM realmente guarde datos.

1. Test de Service con Mock

Archivo: internal/service/service_seguridad_test.go

Este archivo prueba la logica de negocio de SeguridadService. No usa HTTP ni base de datos real.

Mock del repositorio de seguridad

Lineas 17-22

```
type mockSeguridadRepo struct {  
    crearAccesoLlamado bool  
    accesoRecibido      models.AccesoCliente  
}
```

Aqui creo un mock del repositorio de seguridad. La variable crearAccesoLlamado sirve para saber si el service intento guardar un acceso. En este test espero que, si el cliente no tiene pago, el service no llegue al repositorio.

Lineas 49-56

```
func (m *mockSeguridadRepo) CrearAcceso(a models.AccesoCliente)
models.AccesoCliente {
    m.crearAccesoLlamado = true
    m.accesoRecibido = a
    a.ID = 1
    return a
}
```

Este metodo simula guardar un acceso. Si se ejecuta, cambia crearAccesoLlamado a true. Si la regla funciona, este metodo no debe ejecutarse cuando el cliente no tiene pago.

Mock de clientes

Lineas 64-72

```
type mockClienteRepo struct {
    clientes map[int]models.Cliente
}

func (m *mockClienteRepo) BuscarClientePorID(id uint) (models.Cliente,
bool) {
    c, ok := m.clientes[int(id)]
    return c, ok
}
```

Uso un mapa para simular que clientes existen. Si el ID esta en el mapa, el cliente existe; si no esta, el service debe rechazarlo.

Mock de pagos

Lineas 82-95

```
type mockPagoRepo struct {
    tienePago bool
}

func (m *mockPagoRepo) ClienteTienePagoEntrada(clienteID uint) bool {
```

```
    return m.tienePago
}
```

Este mock controla si el cliente tiene pago o no. En el test coloco tienePago en false para probar el caso invalido.

Test: cliente sin pago no llega al repositorio

Lineas 101-120

```
func TestCrearAcceso_SinPagoNoLlegaAlRepo(t *testing.T) {
    repo := &mockSeguridadRepo{}
    clientes := &mockClienteRepo{
        clientes: map[int]models.Cliente{
            2: {ID: 2, Nombre: "Luis Pino"},
        },
    }
    pagos := &mockPagoRepo{tienePago: false}

    svc := NewSeguridadService(repo, clientes, pagos)

    _, err := svc.CrearAcceso(2)
    if err != ErrClienteSinAcceso {
        t.Fatalf("se esperaba ErrClienteSinAcceso, se obtuvo: %v", err)
    }

    if repo.crearAccesoLlamado {
        t.Error("CrearAcceso NO debio llegar al repositorio: el cliente
no tiene pago")
    }
}
```

Aqui el cliente existe, pero no tiene pago. Por eso CrearAcceso debe devolver ErrClienteSinAcceso y no debe llamar al repositorio. Esto cumple el requisito del documento porque prueba una regla real y verifica que un dato invalido no se guarde.

2. Test de Handler con httptest y Fake

Archivo: internal/handlers/handlers_seguridad_test.go

Esta capa simula peticiones HTTP sin levantar el servidor real.

Fake en memoria

Lineas 21-24

```
type fakeSeguridadRepo struct {  
    guardavidas []models.Guardavida  
    siguienteID uint  
}
```

Esto es un fake porque si guarda datos, pero en memoria. Guarda guardavidas en un slice, no en SQLite.

Lineas 35-40

```
func (f *fakeSeguridadRepo) CrearGuardavida(g models.Guardavida)  
models.Guardavida {  
    f.siguienteID++  
    g.ID = f.siguienteID  
    f.guardavidas = append(f.guardavidas, g)  
    return g  
}
```

Este metodo simula una base de datos: incrementa el ID, asigna el ID al guardavida y lo guarda en memoria. Asi pruebo el handler sin tocar la base real.

Router de prueba y token

Lineas 119-154

```
func montarRouterPrueba(t *testing.T) (router chi.Router, tokenValido  
string) {  
    hash, err := bcrypt.GenerateFromPassword([]byte("clave123"),  
    bcrypt.DefaultCost)  
    usuarios := &fakeUsuarioRepo{...}  
    authSvc := service.NewAuthService(usuarios)  
    token, _, err := authSvc.Login("admin@prueba.com", "clave123")  
  
    seguridadSvc := service.NewSeguridadService(&fakeSeguridadRepo{,
```

```
&fakeClienteRepo{}, &fakePagoRepo{}}
    srv := NewServer(seguridadSvc, nil, nil, authSvc)

    r := chi.NewRouter()
    r.Route("/api/v1", func(r chi.Router) {
        r.Group(func(r chi.Router) {
            r.Use(middleware.Auth(authSvc))
            r.Route("/guardavidas", func(r chi.Router) {
                r.Get("/", srv.ListarGuardavidas)
                r.Post("/", srv.CrearGuardavida)
            })
        })
    })
    return r, token
}
```

Esta funcion monta solo las rutas necesarias para el test. Tambien genera un token real usando AuthService y protege las rutas con middleware.Auth, igual que en produccion.

Test 401 sin token

Lineas 163-174

```
func TestListarGuardavidas_SinToken_Devuelve401(t *testing.T) {
    router, _ := montarRouterPrueba(t)

    req := httptest.NewRequest(http.MethodGet, "/api/v1/guardavidas/",
    nil)
    rec := httptest.NewRecorder()

    router.ServeHTTP(rec, req)

    if rec.Code != http.StatusUnauthorized {
        t.Fatalf("se esperaba 401, se obtuvo %d", rec.Code)
    }
}
```

Creo una peticion GET sin token. Como no envio Authorization, el middleware Auth responde 401 antes de llegar al handler.

Test crear guardavida con token

Lineas 179-215

```
func TestCrearGuardavida_ConToken_CreaYPersisteEnFake(t *testing.T) {
    router, token := montarRouterPrueba(t)

    body, _ := json.Marshal(models.Guardavida{Nombre: "Sofia Loor",
    Turno: "manana"})
    req := httptest.NewRequest(http.MethodPost, "/api/v1/guardavidas/",
    bytes.NewReader(body))
    req.Header.Set("Authorization", "Bearer "+token)
    req.Header.Set("Content-Type", "application/json")
    rec := httptest.NewRecorder()

    router.ServeHTTP(rec, req)

    if rec.Code != http.StatusCreated {
        t.Fatalf("se esperaba 201, se obtuvo %d", rec.Code)
    }
}
```

Este test prueba el camino correcto: envio token valido, creo un guardavida y espero 201 Created. Luego el test lista los guardavidas para confirmar que el fake lo guardo.

3. Test de Repository con GORM y SQLite :memory:

Archivo: internal/storage/storage_seguridad_test.go

Aqui no uso mock ni fake. Uso GORM real contra SQLite en memoria.

Base de datos temporal

Lineas 15-24

```
func abrirDBPrueba(t *testing.T) *gorm.DB {
    t.Helper()
    db, err := gorm.Open(sqlite.Open(":memory:"), &gorm.Config{})
    if err != nil {
        t.Fatalf("no se pudo abrir la base de datos en memoria: %v", err)
    }
}
```

```
}  
if err := db.AutoMigrate(&models.Guardavida{}); err != nil {  
    t.Fatalf("fallo AutoMigrate: %v", err)  
}  
return db  
}
```

Se crea una base de datos temporal con `sqlite.Open(":memory:")`. Esto permite probar GORM real sin modificar `piscina.db`. Al terminar el test, esa base desaparece.

Crear, buscar y listar

Lineas 33-66

```
func TestAlmacenSQLite_CrearYListarGuardavida(t *testing.T) {  
    db := abrirDBPrueba(t)  
    almacen := NuevoAlmacenSQLite(db)  
  
    nuevo := models.Guardavida{  
        Nombre: "Pedro Salazar",  
        Turno: "tarde",  
        Certificado: "Cruz Roja Niv. 1",  
        Activo: true,  
    }  
  
    creado := almacen.CrearGuardavida(nuevo)  
    encontrado, ok := almacen.BuscarGuardavidaPorID(creado.ID)  
    lista := almacen.ListarGuardavidas()  
}
```

Este test crea un guardavida, verifica que GORM le asigne ID, luego lo busca por ID y finalmente lo lista. Si el repositorio no persistiera bien, el test fallaria.

4. Cumplimiento de la actividad

Requisito	Evidencia	Cumple
Service con mock	service_seguridad_test.go	Si

Regla real	CrearAcceso rechaza cliente sin pago	Si
No llega al repo	crearAccesoLlamado queda false	Si
Handler con httptest	handlers_seguridad_test.go	Si
Fake en memoria	fakeSeguridadRepo guarda en slice	Si
401 sin token	TestListarGuardavidas_SinToken_Devuelve401	Si
Repositorio con GORM	storage_seguridad_test.go	Si
SQLite :memory:	sqlite.Open(":memory:")	Si

5. Cierre

En resumen, mi modulo Seguridad cumple con los tres niveles de testing que pide la actividad. En service uso mocks para validar reglas de negocio. En handler uso httptest y un fake en memoria para probar rutas HTTP y el 401. En storage uso GORM con SQLite :memory: para probar persistencia real sin afectar la base del proyecto.